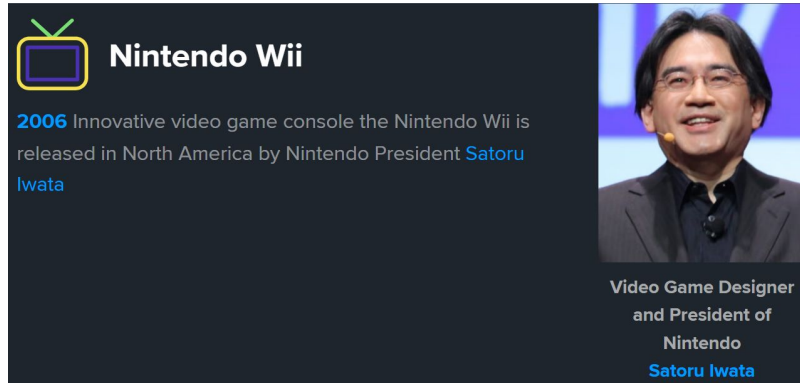


COMS W4156 Advanced Software Engineering (ASE)

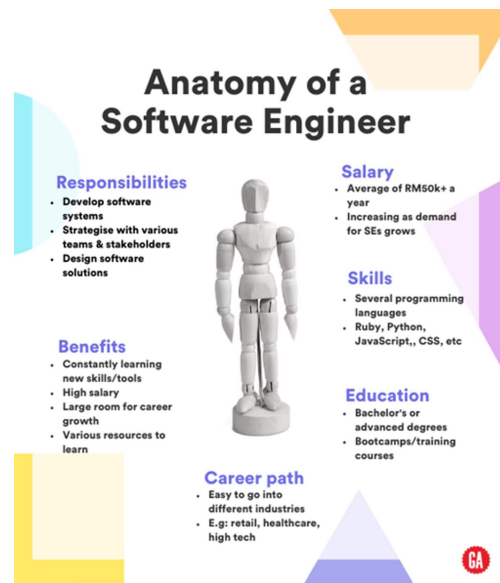
November 19, 2024

On this day ...



Agenda

1. Connecting to DBs: Madeleine Lerner and 100-of-100 team
2. Logging, metrics and observability: Bin Choi and Shivansh Srivastava
3. Mediator
4. Facade
5. Strategy
6. Coming Soon



Madeleine Lerner and 100-of-100 Team Database Demo

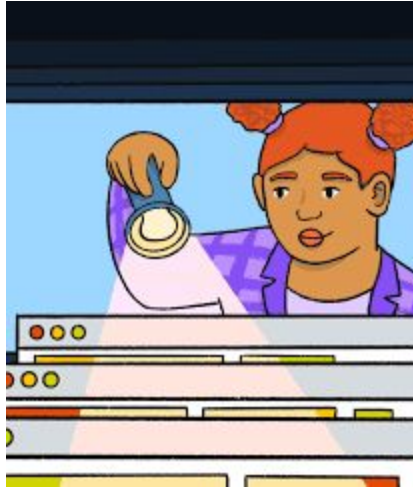
[demo slides - team](#)

[demo slides - leader](#)



Bin Choi and Shivansh Srivastava Monitoring Demo

[demo slides](#)



Agenda

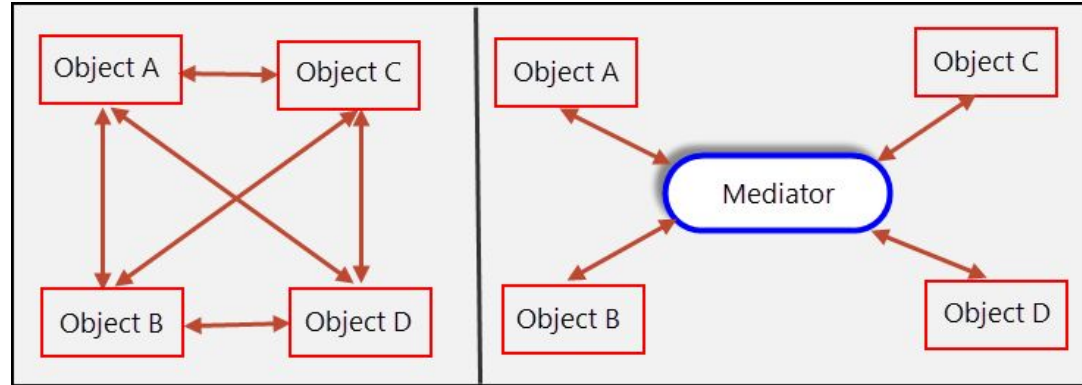
1. Connecting to DBs: Madeleine Lerner and 100-of-100 team
2. Logging, metrics and observability: Bin Choi and Shivansh Srivastava
3. Mediator
4. Facade
5. Strategy
6. Coming Soon



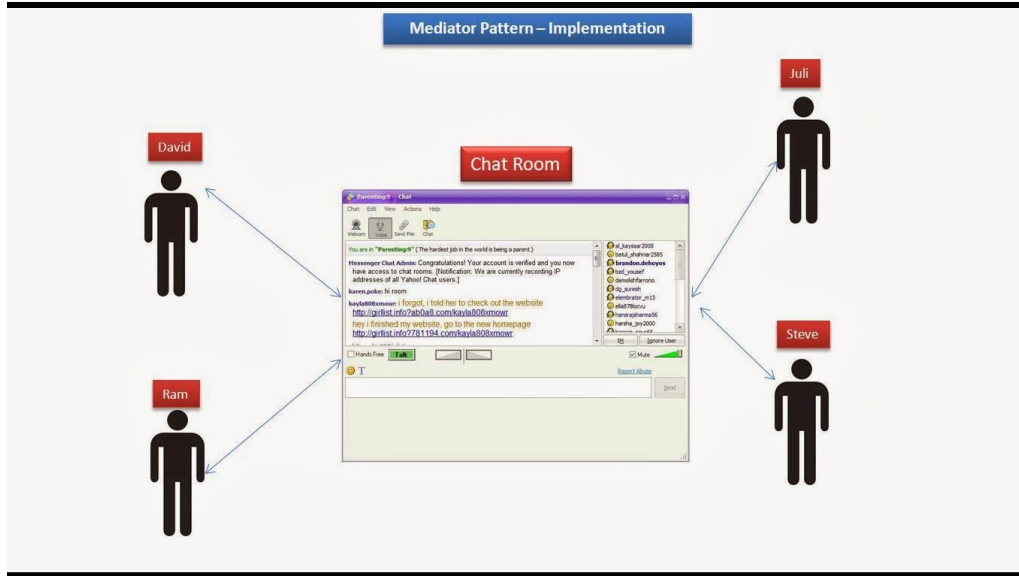
Mediator Pattern

Behavioral design pattern that

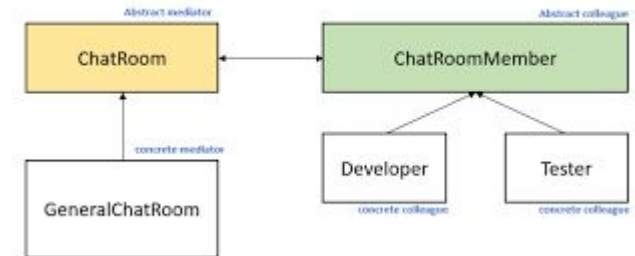
- eliminates mutual (two-way) dependencies among collection of objects,
- restricts direct communications between those objects, and
- forces them to collaborate only via a single mediator object



When to Use Mediator



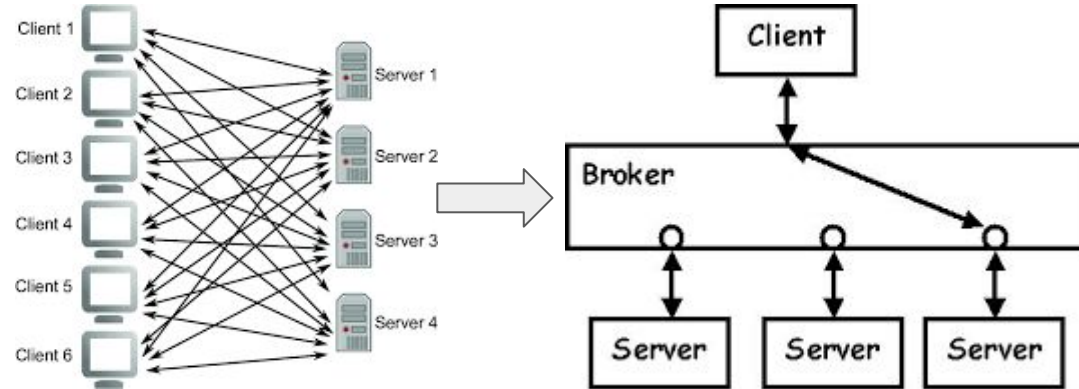
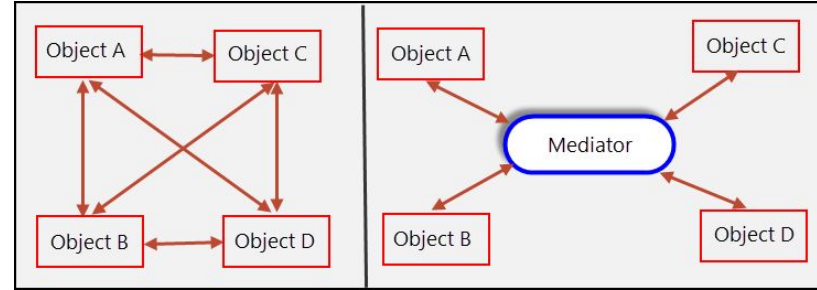
Use when some classes are tightly coupled to several other classes, so they are hard to change - the Mediator pattern lets you extract all the relationships between classes into a separate class, isolating any changes to a specific component



Mediator Pattern vs. Broker Architecture

Related to [broker architecture](#) at object instead of system granularity

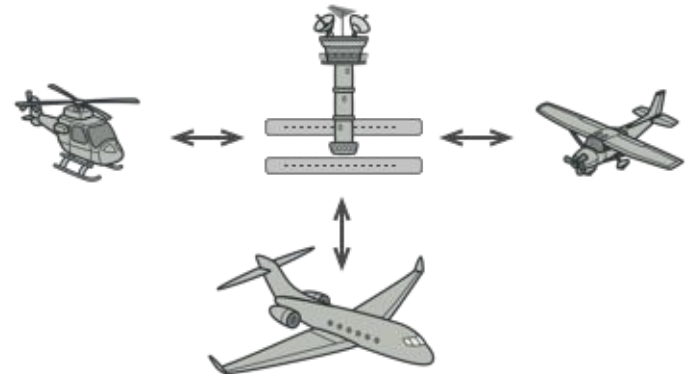
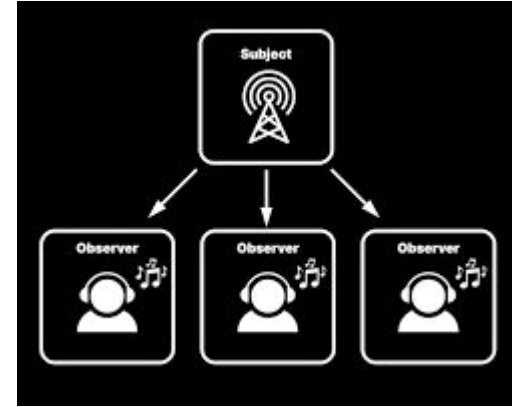
But mediator enables symmetric two-way communication among colleagues rather than request-response from clients to servers



Mediator Pattern vs. Observer Pattern

Both at object granularity

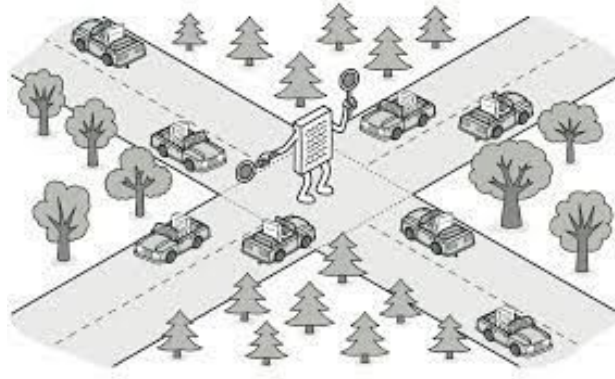
Mediator dependencies are two-way
whereas observer dependencies are one-way



Mediator Pattern Code Examples

[C++](#)

[Java](#)



Agenda

1. Connecting to DBs: Madeleine Lerner and 100-of-100 team
2. Logging, metrics and observability: Bin Choi and Shivansh Srivastava
3. Mediator
4. **Facade**
5. Strategy
6. Coming Soon



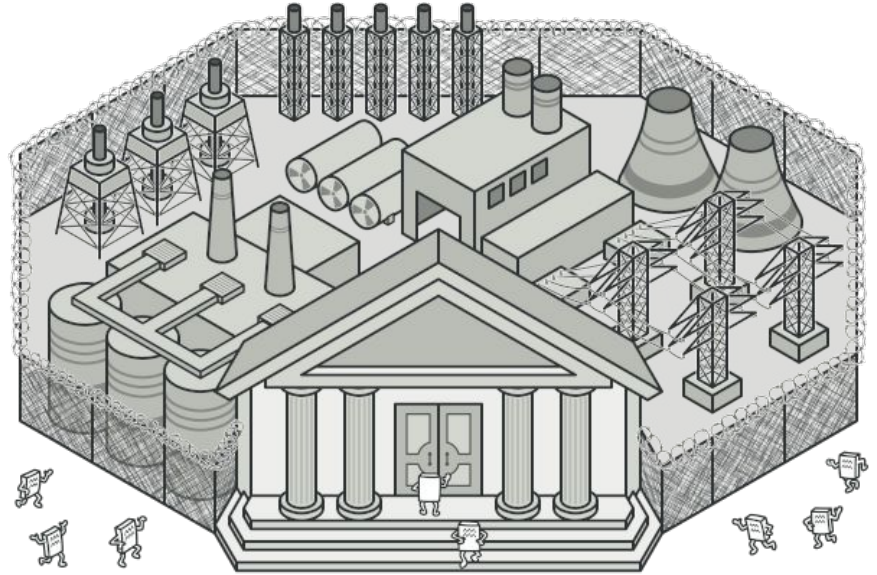
Facade Pattern is Like a Mediator for a Subsystem

Facade is a structural design pattern that provides a simplified interface to a library or other complex set of classes

Structural patterns explain how to assemble objects and classes into larger structures, while keeping the structures flexible and efficient

Facade and Mediator have similar jobs: they try to organize collaboration between lots of tightly coupled classes

Facade defines a simplified interface to a subsystem of objects, but objects *within* the subsystem can still communicate directly



Problem that Facade Addresses

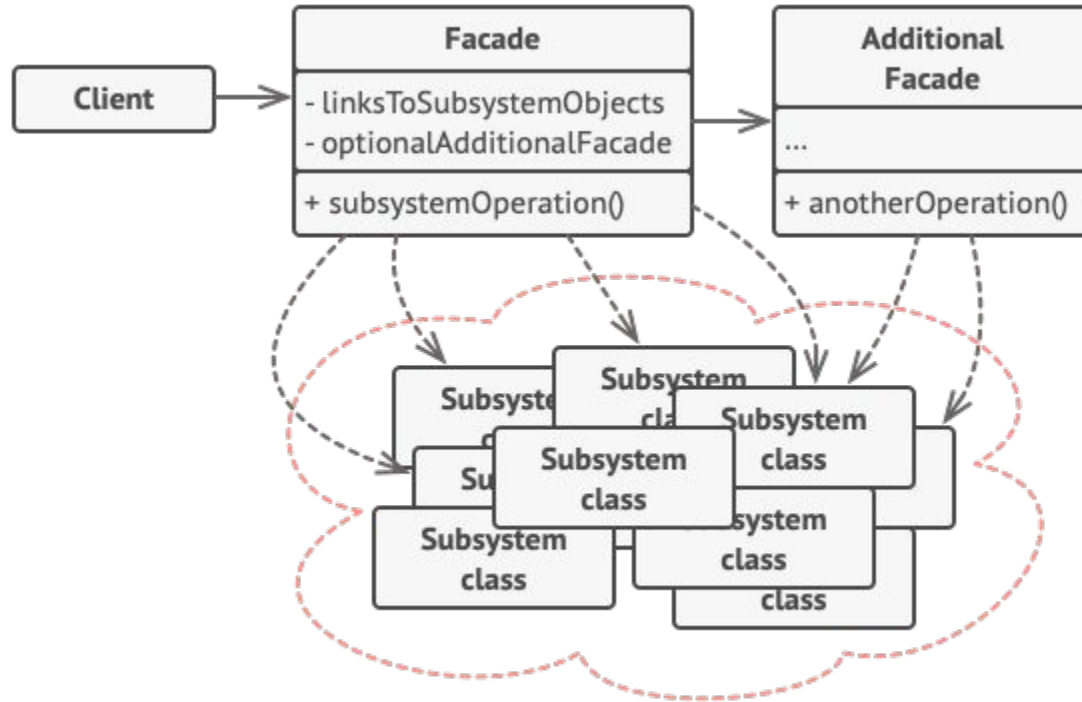


Imagine your code needs to work with a *subset* of some fancy third-party library, and you're not a power-user who needs full control of "everything"

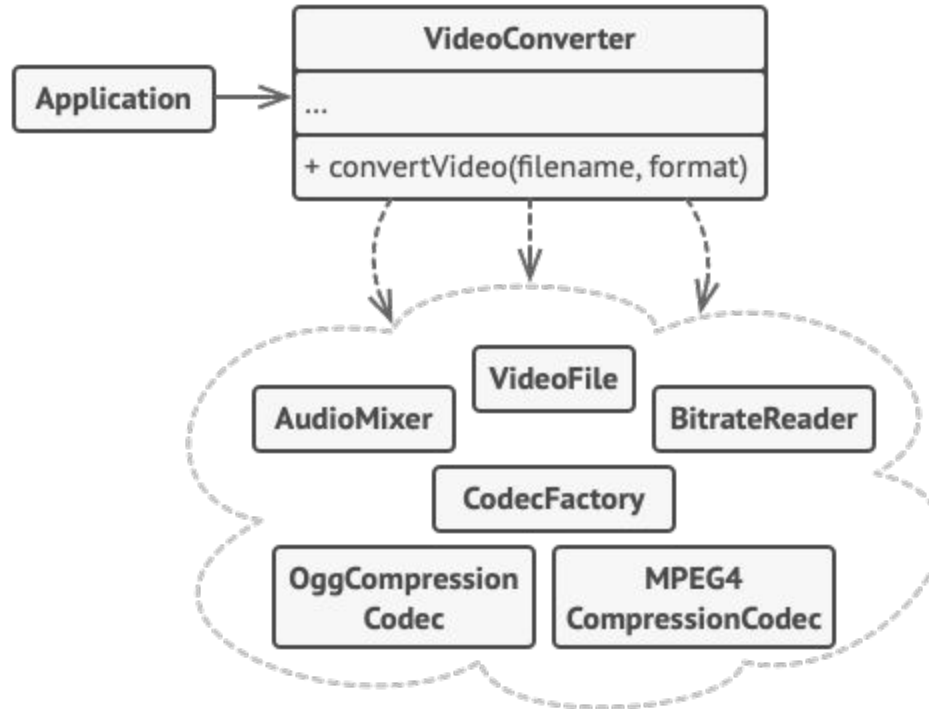
You'd still need to initialize all the third-party objects you use (and possibly others you don't), keep track of dependencies among those objects, execute method calls on different objects in the correct order, supply data in proper format that might be different for different objects, and so on

Your business logic would become tightly coupled to the implementation details of 3rd-party classes

Facade Solution



Facade Example



When to Use Facade

Use the Facade pattern when you need to have a limited but straightforward interface to a complex subsystem

Notice the Facade pattern is presented as the developer adding a facade to *someone else's* library, but alternatively the subsystem could come with multiple facades to choose from

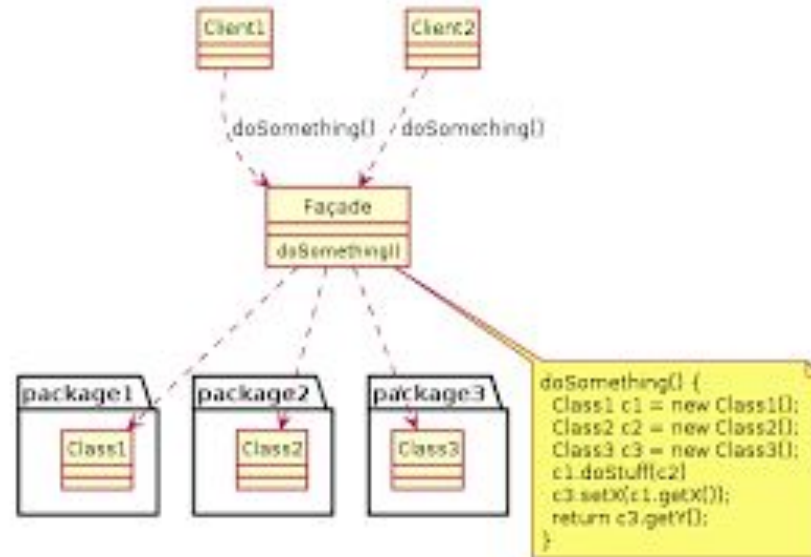
Similarly to applying Mediator, your components become unaware of the individual components within the subsystem so can replace with a different subsystem implementing the same interface



Facade Pattern Code Examples

C++

Java



Agenda

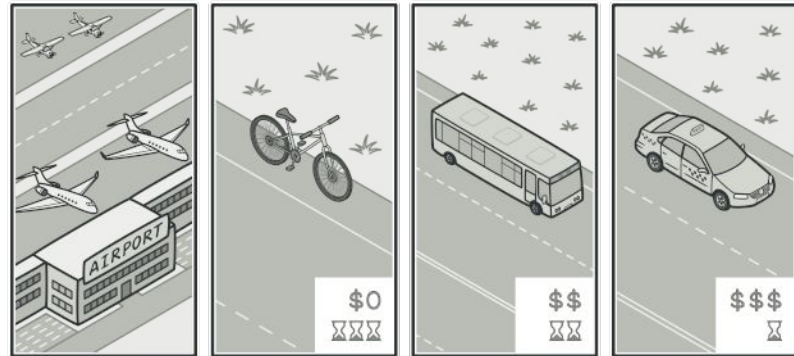
1. Connecting to DBs: Madeleine Lerner and 100-of-100 team
2. Logging, metrics and observability: Bin Choi and Shivansh Srivastava
3. Mediator
4. Facade
5. Strategy
6. Coming Soon



What's the Strategy Pattern?

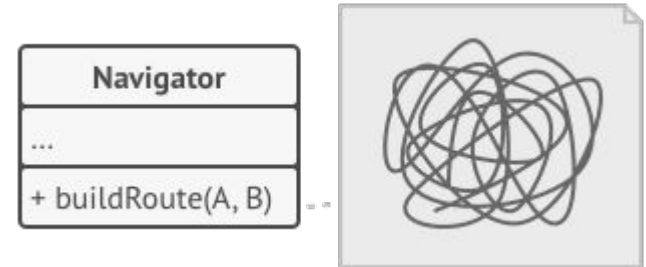
Behavioral design pattern that lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable

May allow swapping algorithms at runtime - e.g., an administrator might switch on the fly between privileged and regular user interfaces (e.g., [mice fake student](#))

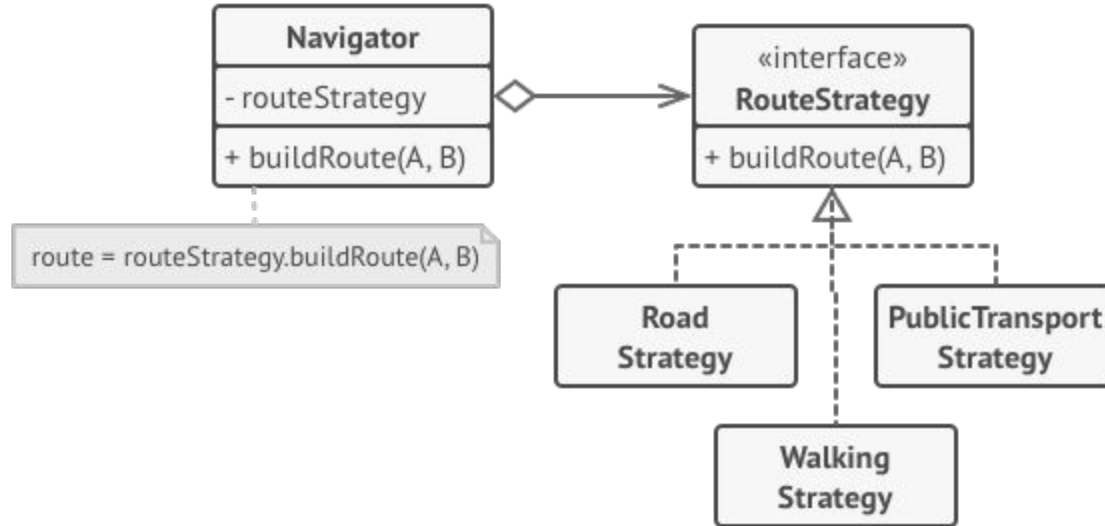


Problem that Strategy Addresses

[Google maps automatic route planning](#)



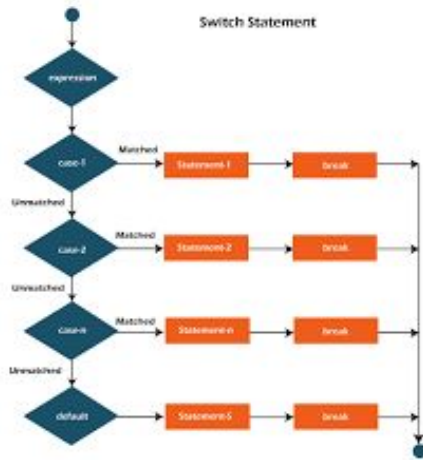
Strategy Solution



When to Use Strategy

Use when you want to use different variants of an algorithm within an object and be able to switch from one algorithm to another - the Strategy pattern lets you indirectly alter the object's behavior at runtime by associating it with different sub-objects that perform specific sub-tasks in different ways

Use when you have a lot of similar classes that only differ in the way they execute some behavior - extract the varying behavior into a separate class hierarchy and combine the original classes into one, thereby reducing duplicate code

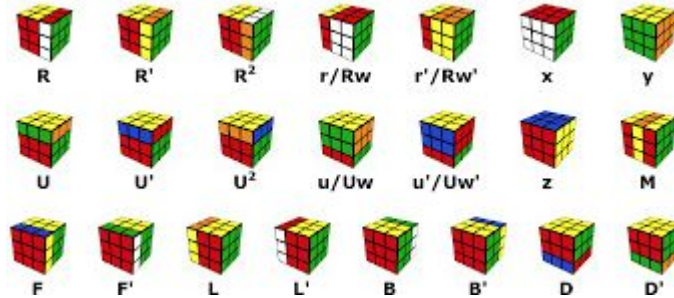


Use when your class has a massive conditional operator that switches between different variants of the same algorithm

Strategy Pattern Code Examples

C++

Java



Many More Design Patterns

The Sacred Elements of the Faith

the holy
origins

107	FM Factory Method
117	PT Prototype
87	AF Abstract Factory
97	BU Builder

the holy
behaviors

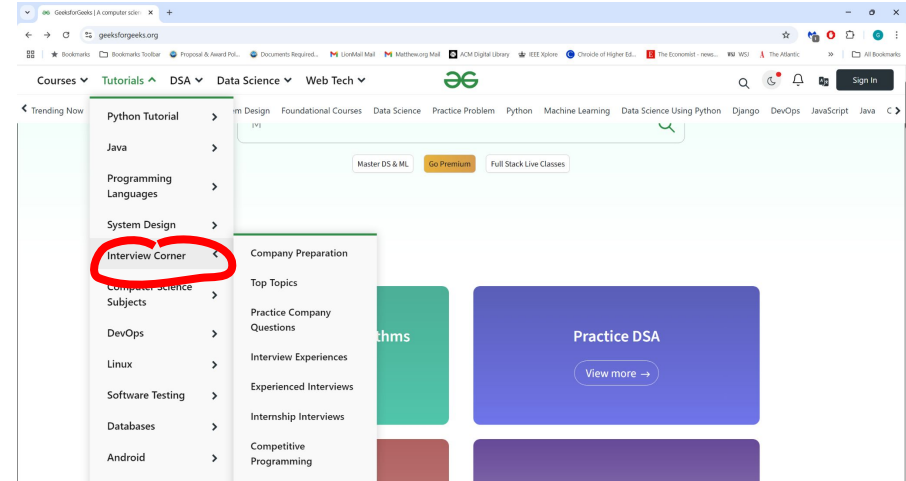
127	S Singleton	233	CR Chain of Responsibility	273	MD Mediator	293	O Observer	243	IN Interpreter	207	PX Proxy	185	FA Facade
325	TM Template Method	273	CD Command	305	ST State	257	IT Iterator	331	V Visitor	195	FL Flyweight	151	BR Bridge

the holy
structures

139	A Adapter	163	CP Composite	175	D Decorator
-----	--------------	-----	-----------------	-----	----------------

Refactoring Guru Design Patterns

Geeks for Geeks Design Patterns



Agenda

1. Connecting to DBs: Madeleine Lerner and 100-of-100 team
2. Logging, metrics and observability: Bin Choi and Shivansh Srivastava
3. Mediator
4. Facade
5. Strategy
6. **Coming Soon**



Rest of Semester

Second Iteration due November 25 (**next week on Monday!**)

Second Iteration Demo due December 6 (Friday the week after Thanksgiving)

Second Assessment Friday December 6 12:01AM to Monday December 9 11:59PM (last day of classes and the day before university "study days")

Demo Day Monday December 16 10AM to 4PM (Monday of finals week)



**ENGLISH: IS "THIS MONDAY"
OR "NEXT MONDAY" THE
CORRECT WAY TO REFER TO
THE VERY NEXT MONDAY
IN THE FUTURE?**

This Thursday

authentication/authorization demo

whatever we did not get to today

more testing topics

as time permits



Next Thursday

Wednesday-Friday are university holidays

But we are supposed to hold class on Tuesday

I plan to cancel Tuesday's class





Ask Me Anything